

Laboratory Notes

Week 6 Lab: ROS Services and Actions

1 Aim

This laboratory aims to explore the concepts of Services and Actions in ROS. It looks at building servers and clients for both Services and Actions, and also includes custom message types. The laboratory will cover the following specific areas:

- Understanding of ROS Services.
- Building a Server and a Client.
- Building a Custom Topic Message and a Custom Server Message.
- Building an Action Message.
- Building an Action Server.
- Analysing how the two nodes interact.

2 Background

Services are an alternative way of inter-node communication in ROS2. When messages are sent via topics, the node creating the data (the publisher) takes an active role and the nodes receiving the data (the subscribers) take a passive role. However, services instead use a call-and-response approach to communication. Whereas topics provide a continual data stream, services only provide data when requested by a client. Services are set-up by a *server*, which defines the data structure of the information that can be requested and announces the name of the service where the data can be requested. When the receiver, called the *client*, wants this data it calls the service with any required arguments and waits for a response.

3 Procedure

We will begin with a simple exercise, using the TurtleSim Simulator, to perform a basic introduction to ROS Services. Launch Turtlesim using the following commands, as per the Week 2 Lab:

```
1 ros2 run turtlesim turtlesim_node
2 ros2 run turtlesim turtle_teleop_key
```

To view the Services that are running and available, use the following command:

```
1 ros2 service list
```

You will notice that multiple Services are running. To determine the message type of a particular service, we can use the following;

```
1 ros2 service type <service>
```

Try:

```
1 ros2 service type /turtle1/teleport_relative
```

We can also see the type of all active services if we include the “show-types” option when using the service list command. This is achieved by including the “-t” option:

```
1 ros2 service list -t
```

If you would like to find all services of a particular type, you can use:

```
1 ros2 service find <type_name>
```

In order to call a service you also need to know the structure of the input arguments for the particular service (message) type. This can be achieved via:

```
1 ros2 interface show <type_name>
```

To then call the service, the following format is used:

```
1 ros2 service call <service_name> <service_type> <arguments>
```

Let’s try some examples of this with TurtleSim. Use the following commands in order, with the output of each preceding command used to determine the correct input for the next. You can use Section 3.3.6 of the Week 2 Lab notes as a guide to how you correctly specify the argument values. Try to get the turtle to move forward by 2 units. Then try to get the turtle to move diagonally.

```
1 ros2 service type /turtle1/teleport_relative
2 ros2 interface show <type_name>
3 ros2 service call /turtle1/teleport_relative <service_type> <arguments>
```

Using a similar procedure, we can also move the turtle to an absolute position. Now have a look at the services “/clear” and “/spawn”. Determine their types and arguments – then call them to see what they do.

Next we will take a look at the range of services available by the various nodes that run during our SLAM and exploration task from the Week 5 Lab. Run the following commands in separate terminals:

```
1 ros2 launch practice_mazes maze_simple_0_world.launch.xml
2 ros2 launch nav2_bringup rviz_launch.py
3 ros2 launch nav2_bringup slam_launch.py
4 ros2 launch ecte477 navigation_launch.py use_sim_time:=true
5 ros2 run ecte477_starter_python maze_explorer
```

Alternatively, you can use the launch file that you wrote during the Week 5 Lab to launch the slam and exploration nodes in one command. Once Gazebo and RViz have finished loading, open a new terminal tab and run “ros2 service list” – you should see a large set of services that are being offered by the various nodes. You can use “ros2 service type” on any of these services to determine the type of the service on offer. Just explore these services a bit, and maybe run “rqt_graph”, to get an overall view of the communication when these nodes are running.

3.1 Custom Topic and Service Messages

3.1.1 Custom Topic Messages

Up until now, we have mainly looked at using existing topic and service messages that are available from the various ROS2 libraries. We can also create our own message types to include within our packages. The steps for this include creating a .msg or .srv file that defines the topic or service message, then making sure that the necessary source code is generated for use in C++ or Python. Start by creating a new package specifically for messages (replace “jeff” with your own name):

```
1 cd
2 cd ros2_ws/src
3 ros2 pkg create jeff_messages
```

Note that we are not specifying a build type here, as this will not be exclusively a python package. Now navigate to your new package and make two new directories called “msg” and “srv”. Using any editor, you should make a new file within the “msg” directory called *Num.msg* and add the following line to the file:

```
1 int64 num
```

This defines a simple message named *jeff_messages/Num* that contains a single member of type int64 named *num*. You could also make a more advanced message, for example one with the following members (this is just an example, there is no need to do this now):

```
1 string first_name
2 string last_name
3 uint8 age
4 unit32 score
```

These files are all that are required to define a custom topic message. You can also include nested types within them, rather than just primitive types. Before being available for use, we will need to run “colcon build” to ensure the build code is generated for our new message type. However, there are some adjustments that we need to do to our configuration files first. In your *package.xml* file, after any existing dependencies, add the following lines:

```
1 <build_depend>rosidl_default_generators</build_depend>
2 <exec_depend>rosidl_default_runtime</exec_depend>
3 <member_of_group>rosidl_interface_packages</member_of_group>
```

Then, in the *CMakeLists.txt* file, after any existing “*find_package*” statements, add the following lines of code:

```
1 find_package(rosidl_default_generators REQUIRED)
2
3 rosidl_generate_interfaces(${PROJECT_NAME}
4     "msg/Num.msg"
5 )
```

If you have an existing “*rosidl_generate_interfaces*” statement, you just need to add “*msg/Num.msg*” to the list of custom message files. Every custom message type you create (whether it be for a topic, a service or an action) will need to be specifically identified here. Save both the *package.xml* and *CMakeLists.txt* files and then run *colcon build* from your *ros2_ws* directory. You can verify if the new message type has been successfully built via (again make sure to substitute your own name for “*jeff*”):

```
1 ros2 interface show jeff_messages/msg/Num
```

3.1.2 Custom Service Messages

Custom services types are defined in a very similar way. Inside the *srv* directory of your *my_messages* package, create a new file called *AddTwoInts.srv* and add the following lines:

```
1 int64 a
2 int64 b
3 ---
4 int64 sum
```

Like with the custom messages, we need to make sure our *package.xml* and our *CMakeLists.txt* files contain the necessary adjustments for our custom service to be built. If you have already made the edits that were detailed in Section 3.1.1, there is nothing further that you need to do for the *package.xml* file; however, you will still need to specifically identify your new service in the *CMakeLists.txt* file. Your edits to the *CMakeLists.txt* file should now look like:

```
1 find_package(rosidl_default_generators REQUIRED)
2
3 rosidl_generate_interfaces(${PROJECT_NAME}
4     "msg/Num.msg"
5     "srv/AddTwoInts.srv"
6 )
```

As with before, save and run *colcon build* (from the *ros2_ws* directory) to generate your service. You can verify this using:

```
1 ros2 interface show my_messages/srv/AddTwoInts
```

3.2 Building a Server and a Client

3.2.1 The Server

In your *my_package* package, create a new file called *add_two_ints_server.py* (this should be placed within the nested *my_package* directory). In this file, add the following code (be sure to change the module name on Line 3 to your own module):

```

1  import rclpy
2  from rclpy.node import Node
3  from jeff_messages.srv import AddTwoInts
4
5  class AddTwoIntsServer(Node):
6      def __init__(self):
7          super().__init__('add_two_ints_server')
8          self.srv = self.create_service(AddTwoInts, 'add_two_ints', self.handle_add_two_ints)
9          print("Ready to add two ints")
10
11     def handle_add_two_ints(self, request, response):
12         response.sum = request.a + request.b
13         print('Returning to client: %s + %s = %s' %(request.a, request.b, response.sum))
14         return(response)
15
16     def main(args=None):
17         rclpy.init(args=args)
18         new_server = AddTwoIntsServer()
19         rclpy.spin(new_server)
20
21     if __name__ == '__main__':
22         main()

```

You can see here that from Section 3.1.2 we have generated a python module called *my_messages.srv* (in my case *jeff_messages.srv*), which we have now included in our import statements. This module contains the necessary message types for using our new service – hence, we import the type that we need: *AddTwoInts*.

To setup the service we need to create it and provide a callback. The callback is run every time a request is received from a client. This is very similar to how a subscriber is defined. The code for setting up the service is within the constructor:

```

6  def __init__(self):
7      super().__init__('add_two_ints_server')
8      self.srv = self.create_service(AddTwoInts, 'add_two_ints', self.handle_add_two_ints)
9      print("Ready to add two ints")

```

The service is called *add_two_ints*, and uses the type *my_messages/srv/AddTwoInts*. The callback specified is called: *handle_add_two_ints()*. The next section of code is the callback method itself:

```

11 def handle_add_two_ints(self, request, response):

```

```

12     response.sum = request.a + request.b
13     print('Returning to client: %s + %s = %s' %(request.a, request.b, response.sum))
14     return(response)

```

As you can see, a service callback takes in two arguments: the service *request* and the service *response*, which are automatically passed whenever the service is called. From these arguments we can access the members of a service type. In the definition of our service message type (Section 3.1.2) we defined two request members: an *int64* called *a* and an *int64* called *b*. We also defined an *int64* response member called *sum*. The callback we have written simply stores the sum of the two request members in the response member, then prints a message and returns the response.

3.2.2 The Client

In your *my_package* package, create a new file called *add_two_ints_client.py* (this should be placed within the nested *my_package* directory). In this file add the following code (be sure to change the module name on Line 4 to your own module):

```

1  import rclpy
2  import sys
3  from rclpy.node import Node
4  from jeff_messages.srv import AddTwoInts
5
6  class AddTwoIntsClient(Node):
7      def __init__(self):
8          super().__init__('add_two_ints_client')
9          self.client = self.create_client(AddTwoInts, 'add_two_ints')
10         while not self.client.wait_for_service(timeout_sec=1.0):
11             print("Waiting for server ..... ")
12         self.request = AddTwoInts.Request()
13
14     def send_request(self, num1, num2):
15         self.request.a = num1
16         self.request.b = num2
17         self.result = self.client.call_async(self.request)
18         rclpy.spin_until_future_complete(self, self.result)
19         return self.result.result()
20
21     def main():
22         if len(sys.argv) == 3:
23             x = int(sys.argv[1])
24             y = int(sys.argv[2])
25         else:
26             print("USAGE: add_two_ints_client x y")
27             sys.exit(1)
28
29     rclpy.init(args=None)

```

```

30
31     new_client = AddTwoIntsClient()
32     response = new_client.send_request(x, y)
33     print("%s + %s = %s" %(x, y, response.sum))
34
35     new_client.destroy_node()
36     rclpy.shutdown()
37
38 if __name__ == '__main__':
39     main()

```

This node is designed to take in two integers as arguments from the command line. It then uses our custom service to request that the server node compute the sum of the two integers. The *main()* function first checks for the correct number of arguments. It then instantiates an object of our client class, before calling the *send_request* method (note that this method is not a callback). The client is setup in the constructor:

```

7     def __init__(self):
8         super().__init__('add_two_ints_client')
9         self.client = self.create_client(AddTwoInts, 'add_two_ints')
10        while not self.client.wait_for_service(timeout_sec=1.0):
11            print("Waiting for server ..... ")
12        self.request = AddTwoInts.Request()

```

It is important that the name and type of the service match what we used in the server node. After setting up the client, the constructor then waits for the server to become available. Finally, it instantiates a request object of our *AddTwoInts* service type. The request to the server is handled by the *send_request* method:

```

14    def send_request(self, num1, num2):
15        self.request.a = num1
16        self.request.b = num2
17        self.result = self.client.call_async(self.request)
18        rclpy.spin_until_future_complete(self, self.result)
19        return self.result.result()

```

This method sets the values of the request members *a* and *b*. It then sends the request to the server and spins until it receives a result, which it returns to the calling function. Note that ROS2 uses future instances when a client sends requests to services or actions. The future instance is essentially a handle to the result of a future event – in this case it references the server response.

3.2.3 Inspecting the Two Nodes

Now build the two nodes using *colcon build*. Don't forget to add entry points for your nodes in your *setup.py* file and ensure you are in the workspace directory before building them:

```

1     cd
2     cd ros2_ws
3     colcon build

```

Then, run the server node using (for example):

```
1 ros2 run my_package add_two_ints_server
```

You can check that the server is active using `ros2 service list`. Now, run the client node using (for example):

```
1 ros2 run my_package add_two_ints_client 4 7
```

Here, we are requesting the server node to compute $4 + 7$.

3.3 ROS Action Servers

3.3.1 Background

The `rclpy.action` library provides a standardised interface for setting up ROS2 actions. An example action is the *NavigateToPose* action used by Nav2. Like services, actions are designed to be done on request and have some kind of response. The difference here is that the execution of the request takes longer to execute, or calculate, so it is designed to have regular feedback on the progress as well as the ability to pre-empt the task to either cancel it or replace it with a new task. Actions also have the flexibility to be interfaced through topics in addition to the service-like interface.

The core of an action is the **action server**. The action server listens for new goals and works on executing them in a callback. It also listens for pre-empt commands so it knows when to quit execution early and optionally swap to a new goal. It can provide continuous feedback on the progress of the goal and finally, publish the result when everything is done. Under the hood, the action server sets up 3 services and publishes on two topics as shown in Figure 1.

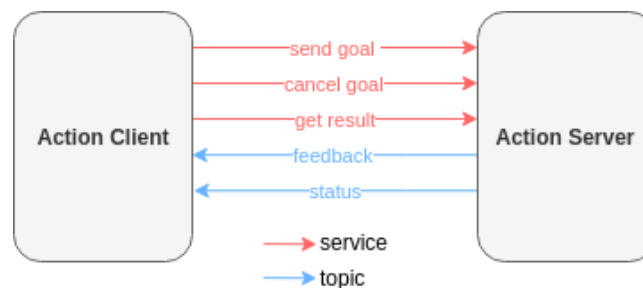


Figure 1: Action Implementation [ros2.org]

3.3.2 Custom Action Messages

Actions are defined by their action messages, which come in three parts and are defined in `.action` files. The **goal** contains the set of input arguments required to start the calculation or execution, the **feedback** defines what messages are sent back to the client as progress is made, and the **result** is the message sent upon completion of the goal.

Let's consider an action where we request a server to calculate a sequence of Fibonacci numbers. The goal will be the order of the sequence to calculate, the feedback will be the in-progress sequence, and the result will

be the final sequence. Inside your *my_messages* package, make a new directory called *action* and in that create a file *Fibonacci.action* containing the following:

```

1  #goal definition
2  int32 order
3  ---
4  #result definition
5  int32[] sequence
6  ---
7  #feedback
8  int32[] sequence

```

Like with our previous custom messages (for topics and services), we need to make sure our *package.xml* and our *CMakeLists.txt* files contain the necessary adjustments for our custom action type to be built. Along with the edits that were made in Section 3.1.1, we also need to add an extra dependency for *action_msgs* to our *package.xml* file:

```

1  <build_depend>roscpp</build_depend>
2  <exec_depend>roscpp</exec_depend>
3  <member_of_group>roscpp</member_of_group>
4
5  <depend>action_msgs</depend>

```

We also need to identify our new action in the *CMakeLists.txt* file. Your edits to the *CMakeLists.txt* file should now look like:

```

1  find_package(roscpp REQUIRED)
2
3  roscpp_generate_interfaces(${PROJECT_NAME}
4      "msg/Num.msg"
5      "srv/AddTwoInts.srv"
6      "action/Fibonacci.action"
7  )

```

As with before, save and run *colcon build* (from the *ros2_ws* directory) to generate your action. You can verify this using:

```

1  ros2 interface show my_messages/action/Fibonacci

```

3.3.3 Writing an Action Server

Now, we will develop an action server that takes fibonacci goals and calculates the sequence. In your *my_package* package, create a new file called *fibonacci_server.py* (as always this should be placed within the nested *my_package* directory). Now add the following code to the new file (be sure to change the module name on Line 5 to your own module):

```

1  import rclpy
2  import time

```

```

3  from rclpy.node import Node
4  from rclpy.action import ActionServer
5  from jeff_messages.action import Fibonacci
6
7  class FibonacciServer(Node):
8      def __init__(self):
9          super().__init__('fibonacci_server')
10         self.act_srv = ActionServer(self, Fibonacci, 'gen_fibonacci_seq', self.fib_callback)
11         self.feedback = Fibonacci.Feedback()
12         self.result = Fibonacci.Result()
13
14     def fib_callback(self, goal):
15         goal.succeed()
16         print("Starting ROS Fibonacci Server Module")
17         self.feedback.sequence = []
18         self.feedback.sequence.append(0)
19         self.feedback.sequence.append(1)
20         self.get_logger().info('Creating Fibonacci sequence of order %s with seeds %s, %s' %(goal.request.order, self.
feedback.sequence[0], self.feedback.sequence[1]))
21
22         for i in range(1, goal.request.order):
23             self.feedback.sequence.append(self.feedback.sequence[i-1] + self.feedback.sequence[i])
24             goal.publish_feedback(self.feedback)
25             time.sleep(1)
26
27         self.get_logger().info('Succeeded')
28         self.result.sequence = self.feedback.sequence
29         return self.result
30
31 def main(args=None):
32     rclpy.init(args=args)
33     new_action_server = FibonacciServer()
34     rclpy.spin(new_action_server)
35
36 if __name__ == '__main__':
37     main()

```

Notice in the import statements that we again need to import our new custom message type. Also, for action servers we need to import the *ActionServer* class from the *rclpy.action* library – this allows us to setup an *ActionServer* object. Let's take a look at the constructor:

```

8  def __init__(self):
9      super().__init__('fibonacci_server')
10     self.act_srv = ActionServer(self, Fibonacci, 'gen_fibonacci_seq', self.fib_callback)
11     self.feedback = Fibonacci.Feedback()
12     self.result = Fibonacci.Result()

```

Here we instantiate an object of the `ActionServer` class – in the arguments we need to provide the name of the node (`self`), the action type (`Fibonacci`), the action name (`gen_fibonacci_seq`) and the callback to run when it receives a goal request (`fib_callback`). We also instantiate *feedback* and *result* objects of our `Fibonacci` action type.

Next we have the callback, which in this case consists of three main sections. The first section calls the *succeed()* method of the goal object (which is just to let the system know that we received the goal), it then initialises our feedback object and prints a couple of messages. The second section of our callback is a processing loop to construct our Fibonacci sequence – this loops up to the order of the sequence and appends the next Fibonacci number to our feedback object on each iteration. We also publish the feedback on each iteration. There is a 1 second sleep included to make our loop process at a rate of 1 Hz. In the third section of our callback we set the result object to be the final Fibonacci sequence and then return the result.

```

14     def fib_callback(self, goal):
15         goal.succeed()
16         print("Starting ROS Fibonacci Server Module")
17         self.feedback.sequence = []
18         self.feedback.sequence.append(0)
19         self.feedback.sequence.append(1)
20         self.get_logger().info('Creating Fibonacci sequence of order %s with seeds %s, %s' %(goal.request.order, self.
feedback.sequence[0], self.feedback.sequence[1]))
21
22         for i in range(1,goal.request.order):
23             self.feedback.sequence.append(self.feedback.sequence[i-1] + self.feedback.sequence[i])
24             goal.publish_feedback(self.feedback)
25             time.sleep(1)
26
27         self.get_logger().info('Succeeded')
28         self.result.sequence = self.feedback.sequence
29         return self.result

```

Note that we could have also included a cancel callback in our Action Server, to deal with cancel goal requests from the client – these are not mandatory, but can be included if you wish to provide the capability of accepting or denying cancel requests.

3.3.4 Writing an Action Client

Next we will develop a simple client to use our server. Again, in your *my_package* package, create a new file called *fibonacci_client.py* (as always this should be placed within the nested *my_package* directory). Now add the following code to the new file (remember to change the name of the module on Line 4 to your own module):

```

1     import rclpy
2     from rclpy.node import Node
3     from rclpy.action import ActionClient
4     from jeff_messages.action import Fibonacci

```

```

5
6 class FibonacciClient(Node):
7     def __init__(self):
8         super().__init__('fibonacci_client')
9         self.act_client = ActionClient(self, Fibonacci, 'gen_fibonacci_seq')
10
11     def send_goal(self):
12         print("Starting ROS Fibonacci Client Module")
13         goal_msg = Fibonacci.Goal()
14         goal_msg.order = 20
15         self.act_client.wait_for_server()
16         send_goal_to_server = self.act_client.send_goal_async(goal_msg)
17         send_goal_to_server.add_done_callback(self.goal_response_callback)
18
19     def goal_response_callback(self, future):
20         goal_handle = future.result()
21         get_result = goal_handle.get_result_async()
22         get_result.add_done_callback(self.final_result_callback)
23
24     def final_result_callback(self, result_future):
25         final_result = result_future.result().result
26         self.get_logger().info('Result: {}'.format(', '.join([str(n) for n in final_result.sequence])))
27         rclpy.shutdown()
28
29     def main(args=None):
30         rclpy.init(args=args)
31         new_action_client = FibonacciClient()
32         new_action_client.send_goal()
33
34         rclpy.spin(new_action_client)
35
36 if __name__ == '__main__':
37     main()

```

The imports are almost identical to that of our server; however, note that we import the *ActionClient* class from the *rclpy.action* library (instead of *ActionServer*). The constructor simply instantiates an object of the *ActionClient* class. In the arguments for the *ActionClient* object we need to provide the node, the action type and the action name (both action type and name need to be identical to that of our server).

The main method of our action client is the *send_goal* method:

```

11 def send_goal(self):
12     print("Starting ROS Fibonacci Client Module")
13     goal_msg = Fibonacci.Goal()
14     goal_msg.order = 20
15     self.act_client.wait_for_server()
16     send_goal_to_server = self.act_client.send_goal_async(goal_msg)

```

```
17 send_goal_to_server.add_done_callback(self.goal_response_callback)
```

In this method we instantiate a *Goal* object of our Fibonacci action type. We then set the *order* member to our required sequence order (in this case: 20). After waiting for the server to become available, the goal is then sent to the server. It is here where things get a little tricky with action clients. The variable we have set when sending the goal (Line 16) is a future instance. This particular future instance is not attached to the overall completion of our goal, but rather the server's acceptance of our goal. Hence, the future instance completes once the server has either accepted or rejected our goal. In Line 17 we are providing a callback to run once the future instance completes.

Lines 19 to 27 contain two callbacks to handle responses from the server:

```
19 def goal_response_callback(self, future):
20     goal_handle = future.result()
21     get_result = goal_handle.get_result_async()
22     get_result.add_done_callback(self.final_result_callback)
23
24 def final_result_callback(self, result_future):
25     final_result = result_future.result().result
26     self.get_logger().info('Result: {}'.format(', '.join([str(n) for n in final_result.sequence])))
27     rclpy.shutdown()
```

The first callback handles the acceptance future instance. We first set a handle to the result of the future instance – we could also check at this point whether or not our goal was accepted; however, in this code we have assumed acceptance. We then send a request to get the final result of our action. Again, the variable we have attached to this request is another future instance (Line 21). This future instance will complete when the goal completes – as this may take a while, we simply include a callback to run once the future instance completes.

The second callback handles the result future instance. Here we obtain the final action result from the future instance and output a message with the final result. The use of future instances takes a little while to master; however, they provide a very neat mechanism for asynchronously dealing with requests that may take a while to complete.

3.3.5 Running the Action Server and Client

Now build the two nodes using *colcon build*. Don't forget to add entry points for your nodes in your *setup.py* file and ensure you are in the workspace directory before building them:

```
1 cd
2 cd ros2_ws
3 colcon build
```

Then, run the server node using (for example):

```
1 ros2 run my_package fibonacci_server
```

You can check that the server is active using *ros2 action list*. Now, run the client node using (for example):

```
1  ros2 run my_package fibonacci_client
```

The server should take around 20 seconds to complete the sequence and return the result. Note that our client did not include any code to request the feedback from the server. We could have provided a callback for the feedback within our client. However, we can also see the server feedback by subscribing to a special ‘hidden’ topic that is automatically setup by the action server. Type the following:

```
1  ros2 topic list --include-hidden-topics
```

In the resulting list you should see two topics related to our action server – one relating to feedback and another relating to status. Try running the client again, while echoing the feedback in a separate terminal:

```
1  ros2 topic echo /gen_fibonacci_seq/_action/feedback
```

In the terminal where you are running the echo command you should now see the feedback provided by our server while performing the action.

4 Summary

In today’s laboratory we have explored the concepts of Services and Actions in ROS. We first looked at making custom message types for both Topics and Services. We then built an example Service, including both the server and client, to add two integers upon request. Finally, we built an example ROS Action, including both server and client, to produce the Fibonacci sequence of a given order (while providing feedback). Services and Actions are used extensively in ROS2 (especially Services) for sophisticated two-way communication that cannot be handled via simple Topics.